PDF Extraction Pipeline Assessment Report

Supraj Sudarshan Gijre

Overview

Built a generalizable PDF extraction pipeline that converts mixed-content PDFs into LLM-friendly Markdown. Validated by correctly answering both assessment questions from the extracted output alone, no reference to the original PDF needed.

Core Design: Per-Page Classification

Every page is independently classified and routed dynamically. This means a single PDF can have any mix of content types in any order and each page is handled correctly without configuration or hardcoded rules.

Route	Trigger	Handler
Digital	Extractable text ≥ 50 chars, no font corruption	pdfplumber + PyMuPDF
Font-Corrupt	Control chars > 1% of page text	Strip-split → Mistral OCR (strips handle both API token limits and dense table truncation)
Image/Scanned	Extractable text < 50 chars	OSD → OpenCV → Mistral OCR

Page 1 : Font-Corrupt Landscape Table

PyMuPDF and pdfplumber both failed silently on this page as the PDF embeds fonts without Unicode mapping tables, producing raw control characters (\x14, \x16) instead of text. Detected by computing control character ratio on the full page text (ratio was 20%, threshold is 1%). Attempted first: Pytesseract OCR. Poor results on a dense 27 column/ 59 rows analytical table as it treats the page as unstructured text and loses all column alignment.

Solution:

- Render page to image
- Detect row count via pdfplumber's vector graphics table finder (immune to font corruption)
- Split into 15-row horizontal strips (strips handle both free API token limits and dense table truncation)
- Send each strip to Mistral OCR with column headers

- Merge strips into one clean Markdown table. Successfully extracted all 59 rows × 27 columns.

Page 2 : Clean Digital Table

Straightforward. pdfplumber uses PDF vector primitives (line segments, rectangles) to detect table structure, producing a perfectly reconstructed Markdown table. PyMuPDF extracts non-table text in reading order via bounding box sorting.

Page 3 Site Map with Floating Tables

Most complex page. A geographic site map with small PFAS data tables floating at each monitoring well location.

Attempted first: Sent full page to Mistral OCR. Returning only the company footer, The model gave up on the dense spatial layout.

Attempted second: Pytesseract with bounding box reconstruction. Rotation was corrected (270° via OSD detection) but the floating table structure was completely lost, output was a flat blob of text.

Solution:

- Tesseract OSD detects and corrects rotation automatically (no hardcoded angles)
- Adaptive Gaussian thresholding (not global Otsu) critical because left-side tables had lighter borders that global thresholding wiped out entirely
- OpenCV line detection + contour finding with overlap deduplication detects individual table bounding boxes
- Each crop sent separately to Mistral OCR, Data extracted from the tables field in the response (the markdown field returns placeholders for complex tables)

Detected and extracted 12 of 13 floating tables. The 13th had partially rendered borders that survived thresholding but not contour detection.

Output

Markdown with section headers, and <!-- end page N --> boundary markers for LLM citation grounding. Mode labels (digital, vlm_ocr_strips, vlm_ocr_image) provide per-page provenance.

Known Limitations

- Cell highlight colors (regulatory exceedance flags) not preserved noted in output
- 1 of 13 floating tables missed on the site map page, a few table heading missing.
- Mistral OCR free tier has rate limits, implemented strips and retry logic, production needs paid tier [Can detect color as well in Pixtral models]

- **Multi-strip formatting inconsistency** Due to free tier rate limiting each strip is sent to Mistral as an independent image, so formatting choices (e.g. date separators) can vary across strips. Merged header cells spanning multiple columns are also flattened, leaving some column labels blank. Mitigation: sending the entire table image or adding coordinates for cells.

Stack: PyMuPDF · pdfplumber · Mistral OCR · Tesseract [OSD] · OpenCV · Pillow · Python 3.12

Evaluation:

[Questions fed to Claude with only extracted MARKDOWN and no reference pdf]

1. What is the value of Arsenic Dissolved at MW-114S on 02/27/2020

Question 1: What is the value of Arsenic, Dissolved at MW-114S on 02/27/2020?

25 (ug/L)

From Page 1, row `Arsenic, Dissolved`, column `MW-114S`, date collected `02-27-2020`.

2. Show all the values of Arsenic with Monitoring Well Information and Dates

Question 2: All values of Arsenic with Monitoring Well Information and Dates

From Page 1 (date: 02/25/2020 – 02/27/2020):

Analyte	MW-100S	MW-101S	MW-102S	MW-103S	MW-104S	MW-105S	MW-106SR	MW-107S	MW-108S	MW-109	MW-110S
Arsenic, Dissolved	<0.002	<0.002	<0.002	4	<0.002	<0.002	<0.002	<0.002	0.257	9	0.518
Arsenic	<0.002	<0.002	<0.002	46	<0.002	0.316	1.173	0.465	<0.002	69	2

Page 1 — Date range: 02/25/2020 – 02/27/2020

V- !S	MW- 103S	MW- 104S	MW- 105S	MW- 106SR	MW- 107S	MW- 108S	MW- 109	MW- 110S	MW- 111S	MW- 112S	MW- 113S	MW- 114S
002	4	<0.002	<0.002	<0.002	<0.002	0.257	9	0.518	<0.002	13	1.511	**25**
002	46	<0.002	0.316	1.173	0.465	<0.002	69	2	0.998	41	3	62

Page 2 — Date: 5/29/18

Analyte	MW-103S	MW-111S (Field Dupe = MW-102S)
Arsenic, Dissolved	1.41	23
Arsenic	9	60

All other wells on 5/29/18 show no detected values (DRY or NS conditions at MW-100S, MW-104S, MW-106SR, MW-108S, MW-109S, MW-110S).

Running the Pipeline

1. `python -m venv env`
2. Activate the environment:
   - Windows: `env\Scripts\activate`
   - Mac/Linux: `source env/bin/activate`
3. `pip install -r requirements.txt`
4. Install Tesseract binary and add to PATH:
   - Windows: Download installer from https://github.com/UB-Mannheim/tesseract/wiki — check "Add to PATH" during install
   - Mac: `brew install tesseract`
   - Ubuntu: `sudo apt install tesseract-ocr`
   - Full installation guide: https://tesseract-ocr.github.io/tessdoc/Installation.html
5. Add your Mistral API key to a `.env` file:
[Already added and set to expire on 20 March 2026]
   `MISTRAL_API_KEY=your_key_here`
6. Run:
   `python run_pipeline.py report.pdf`

In case you want to visualize the .md file use the following link:
https://markdownlivepreview.com/